



Python, FastAPI & Enterprise API Standards

Design, Build, Secure, Observe & Certify Production APIs

Special Focus: Observability & SLO API Testing — CMM Maturity Levels 1–5

Training Notes

· FastAPI · Starlette · Pydantic · pytest · Prometheus · OpenTelemetry

by

Dr. D.S.R. Murthy_{M.Tech(IT)}

dsrmurthysoftware@gmail.com

Cell: 9848011641

Table of Contents

Python, FastAPI & REST Fundamentals

Module 1 — Python Fundamentals for API Development	3
Module 2 — FastAPI Architecture & Core Concepts	.
Module 3 — Request Handling, Validation & Error Responses	.
Module 4 — Authentication, Pagination & Health Checks	.

API Design Principles & Operational Best Practices

Module 5 — API Design Principles & Responsibility Framework	.
Module 6 — API Operational Best Practices	.

Contract-First Design & API Standards

Module 7 — Contract-First Design & OpenAPI	.
Module 8 — Resource Modeling & URL Design	.
Module 9 — HTTP Semantics, Status Codes & Error Design	.
Module 10 — Versioning, Deprecation & API Lifecycle	.

API Quality, Testing & Operational Maturity

Module 11 — The Tiered Definition of Done	.
Module 12 — API Testing: Compliance & Negative Testing	.
Module 13 — Observability: Logging, Metrics & Tracing	.
Module 14 — Advanced Patterns & Long-Term Maturity	.

Appendix

Appendix A — Observability & SLO API Testing: CMM Levels 1–5 (Hands-on Exercises)	.
Appendix B — Quick Reference Tables	.

Python, FastAPI & REST Fundamentals

Build working, compliant API endpoints from the ground up.

Module 1 — Python Fundamentals for API Development

1.1 Virtual Environments & Dependency Management

Every FastAPI project should be isolated from the system Python to keep dependency versions reproducible across dev, CI, and production.

```
# create + activate a virtual environment
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # macOS/Linux

# install and freeze dependencies
pip install fastapi uvicorn[standard] pydantic pytest
pip freeze > requirements.txt
```

Best practice: pin exact versions in `requirements.txt` (or use `poetry` / `pip-tools`) so CI and production install the identical dependency graph you tested against.

1.2 Type Hints and What They Enable

Type hints are not decoration — FastAPI reads them at runtime to build validation, serialization, and the OpenAPI schema.

```
def add_item(name: str, quantity: int, price: float = 0.0) -> dict:
    return {"name": name, "quantity": quantity, "price": price}
```

Type hints enable three things FastAPI depends on:

- **Validation** — incoming JSON is coerced/validated against the hinted type.
- **Serialization** — return values are converted to JSON using the declared response type.
- **Documentation** — the type becomes part of the auto-generated OpenAPI schema.

1.3 Pydantic Models for Request & Response Schemas

```

from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime

class OrderCreate(BaseModel):
    customer_id: str = Field(..., min_length=1)
    quantity: int = Field(..., gt=0)
    notes: Optional[str] = None

class OrderRead(BaseModel):
    id: str
    customer_id: str
    quantity: int
    created_at: datetime

class Config:
    from_attributes = True  # build model from ORM objects

```

Separate **Create**, **Update**, and **Read** models — never reuse the same Pydantic model for input and output. It couples your public contract to your internal storage shape.

1.4 Async vs. Synchronous Functions

Use <code>async def</code>	Use <code>def</code>
I/O-bound work: awaiting a DB driver, HTTP client, queue	CPU-bound work, or calling blocking/sync libraries
Runs on the event loop directly	FastAPI runs it in a thread pool automatically

```

async def get_order(order_id: str):
    return await db.fetch_one(order_id)  # non-blocking

def compute_report(data: list):
    return heavy_cpu_bound_transform(data)  # offloaded to threadpool by FastAPI

```

Pitfall: calling a blocking library (e.g. `requests`, blocking DB driver) inside `async def` blocks the entire event loop for every concurrent request.

1.5 Organizing Code Across Files & Modules

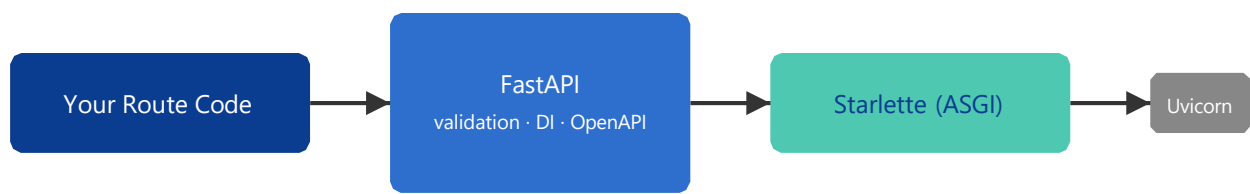
```

app/
├─ main.py           # app instance, routers, middleware
├─ api/
│  └─ routers/
│     └─ orders.py
│     └─ health.py
├─ models/           # Pydantic schemas
├─ services/         # business logic
├─ dependencies.py   # shared Depends() callables
└─ core/
   └─ config.py
   └─ logging.py

```

1.6 Starlette — the ASGI Foundation

FastAPI is built directly on top of **Starlette**, a lightweight ASGI framework. FastAPI adds data validation, serialization, dependency injection, and OpenAPI generation on top of Starlette's routing, middleware, and request/response primitives.



Module 2 — FastAPI Architecture & Core Concepts

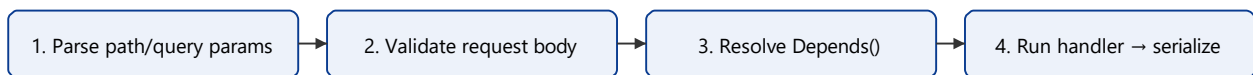
Understand how FastAPI handles a request from arrival to response.

2.1 FastAPI as an ASGI Framework

ASGI (Asynchronous Server Gateway Interface) lets a single framework handle HTTP, WebSockets, and background lifespan events asynchronously. Uvicorn (the ASGI server) receives the socket connection and calls into the Starlette/FastAPI ASGI `app(scope, receive, send)` callable.



Inside the path function, FastAPI:



2.2 Routing, Path Parameters & Query Parameters

```
app = FastAPI()

@app.get("/orders/{order_id}")
async def get_order(
    order_id: str,                    # path parameter - required
    include_items: bool = False,     # query parameter - optional
    limit: int = Query(20, le=100),  # validated query parameter
):
    ...
```

2.3 Request Body Injection & Response Models

```
@app.post("/orders", response_model=OrderRead, status_code=201)
async def create_order(payload: OrderCreate) -> OrderRead:
    order = await order_service.create(payload)
    return order
```

`response_model` guarantees consumers only ever see the declared output shape — even if your handler accidentally returns extra internal fields.

2.4 Shared Logic with Dependency Injection

```
def get_db() -> Generator:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

async def get_current_user(token: str = Depends(oauth2_scheme)) -> User:
    return decode_and_lookup_user(token)

@app.get("/orders")
async def list_orders(db=Depends(get_db), user: User = Depends(get_current_user)):
    return db.query(Order).filter_by(owner_id=user.id).all()
```

Dependencies compose: `get_current_user` can itself depend on `oauth2_scheme`. FastAPI resolves the whole graph per-request and caches results within a request by default.

2.5 Auto-Generated OpenAPI Contract

Every route's types, models, and status codes are introspected to build `/openapi.json`, rendered as interactive docs at `/docs` (Swagger UI) and `/redoc`. This makes FastAPI naturally contract-adjacent — but as Day 3 covers, the contract should be *designed*, not just emitted as an afterthought.

Module 3 — Request Handling, Validation & Error Responses

Validate input and return consistent, standards-compliant errors on every route.

3.1 Automatic Request Validation

Pydantic validates every field on the incoming body/query/path against your model. A validation failure short-circuits the handler entirely and returns `422 Unprocessable Entity` by default.

```
class OrderCreate(BaseModel):
    quantity: int = Field(..., gt=0, description="Must be positive")
    email: EmailStr
```

3.2 Standard Error Envelope

Never leak raw stack traces or inconsistent shapes. Every error response — validation, business, or system — should share one envelope:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "quantity must be greater than 0",
    "request_id": "a1b2c3d4-...",
    "details": [{"field": "quantity", "issue": "must be > 0"}]
  }
}
```

```
class ApiError(BaseModel):
    code: str
    message: str
    request_id: str
    details: list[dict] | None = None

class ErrorEnvelope(BaseModel):
    error: ApiError
```

3.3 Stable, Specific Error Codes

HTTP status communicates the *category*; the `error.code` string communicates the *specific, stable, machine-readable* reason consumers can branch on (it must not change across releases even if the message text does).

HTTP	error.code	Meaning
400	VALIDATION_ERROR	Malformed / semantically invalid request
401	UNAUTHENTICATED	No/invalid credentials
403	FORBIDDEN	Authenticated but not permitted
404	NOT_FOUND	Resource does not exist (or is hidden for security)

409	CONFLICT	State conflict (duplicate, version mismatch)
429	RATE_LIMITED	Too many requests
500	INTERNAL_ERROR	Unhandled server fault

3.4 Global Exception Handler for Unhandled Errors

```
@app.exception_handler(RequestValidationError)
async def validation_handler(request: Request, exc: RequestValidationError):
    return JSONResponse(
        status_code=422,
        content={"error": {
            "code": "VALIDATION_ERROR",
            "message": "Request failed validation",
            "request_id": request.state.request_id,
            "details": exc.errors(),
        }},
    )

@app.exception_handler(Exception)
async def unhandled_handler(request: Request, exc: Exception):
    logger.exception("unhandled_error", extra={"request_id": request.state.request_id})
    return JSONResponse(
        status_code=500,
        content={"error": {"code": "INTERNAL_ERROR",
            "message": "An unexpected error occurred",
            "request_id": request.state.request_id}},
    )
```

Never return raw exception messages or stack traces to the client in the 500 body — they leak internals and violate log-safety rules (Module 6).

3.5 Request Tracing with a Unique ID per Request

```
import uuid
from starlette.middleware.base import BaseHTTPMiddleware

class RequestIDMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        request_id = request.headers.get("X-Request-ID", str(uuid.uuid4()))
        request.state.request_id = request_id
        response = await call_next(request)
        response.headers["X-Request-ID"] = request_id
        return response

app.add_middleware(RequestIDMiddleware)
```

This ID must flow through: the response header, every log line for that request, and the error envelope — so a consumer's bug report can be traced end-to-end in one grep.

Module 4 — Authentication, Pagination & Health Checks

Secure every endpoint and expose the signals production requires.

4.1 Bearer Token Authentication

```
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

bearer_scheme = HTTPBearer()

async def get_current_user(creds: HTTPAuthorizationCredentials = Depends(bearer_scheme)) -> User:
    token = creds.credentials
    try:
        payload = decode_jwt(token)
    except InvalidTokenError:
        raise HTTPException(status_code=401, detail="UNAUTHENTICATED")
    return load_user(payload["sub"])
```

4.2 Authentication vs. Authorization vs. Existence

Choosing the correct status code matters — collapsing these into one code either leaks information or misleads consumers.

Situation	Correct code	Why
No/invalid token	401 Unauthenticated	We don't know who you are yet
Valid identity, insufficient permission	403 Forbidden	We know who you are; you're not allowed
Resource doesn't exist	404 Not Found	Nothing to authorize against
Resource exists but caller shouldn't know that	404 (not 403)	Avoid confirming existence to unauthorized callers

4.3 Cursor-Based Pagination Envelope

```
{
  "data": [ {"id": "o_123", ...}, {"id": "o_124", ...} ],
  "page": {
    "next_cursor": "eyJpZCI6Im9fMTI0In0=",
    "has_more": true,
    "limit": 20
  }
}
```

```
@app.get("/orders", response_model=OrderPage)
async def list_orders(cursor: str | None = None, limit: int = Query(20, le=100)):
    rows = repo.fetch_page(after=decode_cursor(cursor), limit=limit + 1)
    has_more = len(rows) > limit
    rows = rows[:limit]
    next_cursor = encode_cursor(rows[-1].id) if has_more else None
    return {"data": rows, "page": {"next_cursor": next_cursor, "has_more": has_more, "limit": limit}}
```

Cursor pagination (vs. offset) stays correct even as rows are inserted/deleted between page requests.

4.4 Standardized UTC Timestamp Serialization

```
class OrderRead(BaseModel):
    created_at: datetime

    class Config:
        json_encoders = {datetime: lambda dt: dt.astimezone(timezone.utc).isoformat().replace("+00", "Z")}
```

Always store and emit UTC in ISO-8601 with a trailing **Z** — never emit naive/local timestamps from an API.

4.5 Liveness & Readiness Health Endpoints

```
@app.get("/healthz/live", tags=["health"])
async def liveness():
    # process is up and able to respond – no dependency checks
    return {"status": "ok"}

@app.get("/healthz/ready", tags=["health"])
async def readiness(db=Depends(get_db)):
    # can the app actually serve traffic right now?
    try:
        await db.execute("SELECT 1")
    except Exception:
        raise HTTPException(status_code=503, detail="NOT_READY")
    return {"status": "ready"}
```

Probe	Checks	On failure, orchestrator...
Liveness	Process is running / not deadlocked	Restarts the container
Readiness	DB, cache, downstream deps reachable	Removes pod from load balancer (no restart)

API Design Principles & Operational Best Practices

Learn the enterprise API guidelines and how to apply them every day.

Module 5 — API Design Principles & Responsibility Framework

Understand who owns what between the API team and the infrastructure layer.

5.1 What Makes a World-Class API

- **Predictable** — consistent naming, status codes, and error shapes across every endpoint.
- **Discoverable** — a complete, accurate OpenAPI contract is the source of truth.
- **Secure by default** — authn/authz enforced centrally, never opt-in per route.
- **Observable** — every request is logged, measured, and traceable.
- **Stable** — versioned, with a published deprecation lifecycle.

5.2 The Responsibility Split



API Team Responsibilities (detail)

- Own the OpenAPI contract and keep it in sync with code.
- Emit structured logs and metrics *in the format* the platform expects.
- Implement authentication/authorization decisions (the infra layer only terminates TLS and may enforce coarse network-level rules).
- Define and monitor their own SLIs/SLOs; alert on breaches.

Infrastructure Layer Responsibilities (detail)

- Provide the runtime platform (Kubernetes, load balancer, service mesh).
- Operate the shared observability pipeline (log shipping, metrics scraping, tracing collectors).
- Enforce perimeter security (WAF, network policies, cert rotation).

Shared Contract

SLOs, the OpenAPI spec, the structured-log schema, and trace-header propagation are jointly owned — the API team emits them correctly; the infra team collects, stores, and alerts on them.

Module 6 — API Operational Best Practices

A guided walkthrough of the enterprise API developer handbook, section by section.

6.1 How the Course Reference Documents Work Together

OpenAPI Spec (contract) → Developer Handbook (how to build it) → Tiered Definition of Done (how to prove it) → Test Suite (compliance evidence).

6.2 OpenAPI as the Source of Truth

The spec is written/updated *before* code changes merge. CI should fail the build if the running app's generated schema drifts from the checked-in contract file.

6.3 Resource Modeling & URL Design Rules

- Plural nouns for collections: `/orders`, not `/order`.
- No verbs in paths: `POST /orders/{id}/cancel` is acceptable as a sub-resource action; `/cancelOrder` is not.
- Nest only one level deep where possible: `/orders/{id}/items`, not `/customers/{id}/orders/{id}/items/{id}/notes`.

6.4 HTTP Methods, Status Codes & Error Responses

Method	Semantics	Typical success code
GET	Safe, idempotent, cacheable	200
POST	Not safe, not idempotent (unless idempotency key used)	201 (created) / 200 / 202
PUT	Idempotent full replace	200 / 204
PATCH	Partial update, not guaranteed idempotent	200
DELETE	Idempotent removal	204

6.5 Security & Log Safety Requirements

- Never log tokens, passwords, secrets, or full PII (mask/redact).
- Enforce TLS everywhere; reject plaintext HTTP at the gateway.
- Validate and sanitize all inputs — treat the network as hostile.

6.6 Observability Requirements

Every endpoint must emit: a structured JSON log line per request (with request ID, route, status, duration), request/error/duration metrics scraped by the metrics platform, and a trace span propagated via standard trace headers. Detailed in Module 13.

6.7 Versioning & Deprecation Lifecycle

Breaking changes require a new major version; the old version is marked `Deprecated`, given a sunset date via headers, and retired only after that date passes with monitored consumer migration. Detailed in Module 10.

Contract-First Design & API Standards

Design every API around the contract, not the implementation.

Module 7 — Contract-First Design & OpenAPI

Write the specification before writing any implementation code.

7.1 Why Contract-First Produces Better APIs



Contract-first surfaces design flaws (bad resource shapes, missing error cases) before any code is written, and lets consumer teams start integration against mocked responses immediately.

7.2 OpenAPI 3.0.x File Structure

```
openapi: 3.0.3
info:
  title: Orders API
  version: "1.2.0"
paths:
  /orders/{orderId}:
    get:
      operationId: getOrder
      parameters:
        - name: orderId
          in: path
          required: true
          schema: { type: string }
      responses:
        '200':
          description: Order found
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Order' }
        '404':
          description: Order not found
          content:
            application/json:
              schema: { $ref: '#/components/schemas/ErrorEnvelope' }
components:
  schemas:
    Order: { # ... }
    ErrorEnvelope: { # ... }
```

7.3 Documenting All Response Codes & Headers

Every route must document *every* status it can realistically return (200/201/204, 400, 401, 403, 404, 409, 422, 429, 500) — not just the happy path — plus response headers such as `X-Request-ID`, `Deprecation`, and `Cache-Control`.

7.4 Keeping Contract and Code in Sync

```
# CI step: diff the live app's schema against the checked-in contract
import json, sys
from app.main import app

live_schema = app.openapi()
committed_schema = json.load(open("openapi.json"))
if live_schema != committed_schema:
    sys.exit("openapi.json is stale — regenerate and commit it")
```

Module 8 — Resource Modeling & URL Design

Model the domain correctly before defining any routes.

8.1 Four Standard Resource Types

Type	Example	Notes
Collection	<code>/orders</code>	Plural noun; GET lists, POST creates
Singleton item	<code>/orders/{id}</code>	GET/PUT/PATCH/DELETE one instance
Sub-resource / relationship	<code>/orders/{id}/items</code>	Owned collection scoped to a parent
Controller / action	<code>/orders/{id}/cancel</code> (POST)	Non-CRUD action; used sparingly, never as a verb-only path

8.2 Canonical Identifiers & System Fields

- Every resource has one immutable `id`, opaque to clients (UUID or prefixed string like `ord_9f2a`).
- System fields (`created_at` , `updated_at` , `etag`) are server-controlled and read-only on write.

8.3 No Verbs in URL Paths

Bad	Good
<code>/getOrder?id=1</code>	GET <code>/orders/1</code>
<code>/createOrder</code>	POST <code>/orders</code>
<code>/deleteOrder/1</code>	DELETE <code>/orders/1</code>

8.4 URL Structure & Naming Rules

- Lowercase, hyphen-separated segments: `/purchase-orders` , not `/purchaseOrders` .
- Version at the root: `/v1/orders` .
- Query params for filtering/sorting/pagination only, never for identifying a resource: `/orders?status=open&limit=20` .

Module 9 — HTTP Semantics, Status Codes & Error Design

Use HTTP correctly so consumers can reason about every response.

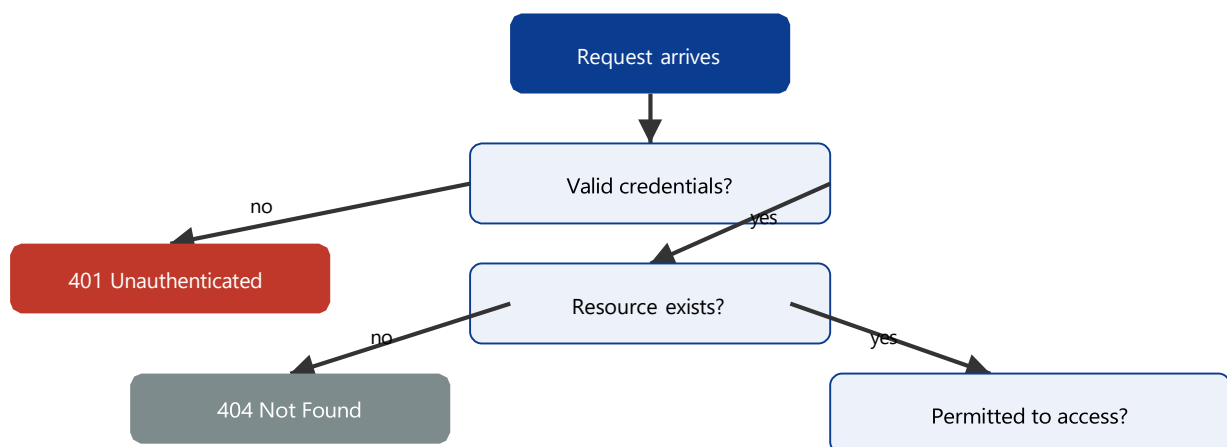
9.1 Safe vs. Idempotent Methods

Method	Safe (no side effects)	Idempotent (repeat = same result)
GET / HEAD	Yes	Yes
PUT / DELETE	No	Yes
POST	No	No (unless idempotency key)
PATCH	No	Not guaranteed

9.2 Status Code Precision

Code	Use exactly when
200	Successful GET/PUT/PATCH with a body
201	Resource created (include <code>Location</code> header)
202	Accepted for async processing (Module 14)
204	Success, no body (DELETE, some PUT)
400	Malformed request the client must fix
422	Well-formed but semantically invalid (validation)
409	Conflicts with current resource state
412	Precondition (If-Match/ETag) failed

9.3 Authentication, Authorization & Existence — Decision Flow



If permitted → return the resource (200). If not permitted → return 403 (or 404 if existence itself must be hidden from unauthorized callers).

9.4 Standard Error Envelope (recap)

See Module 3.2 — the same envelope shape applies everywhere: `{"error": {"code", "message", "request_id", "details"}}`.

9.5 Validation Error Detail

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "2 fields failed validation",
    "request_id": "...",
    "details": [
      {"field": "quantity", "issue": "must be greater than 0"},
      {"field": "email", "issue": "not a valid email address"}
    ]
  }
}
```

Module 10 — Versioning, Deprecation & API Lifecycle

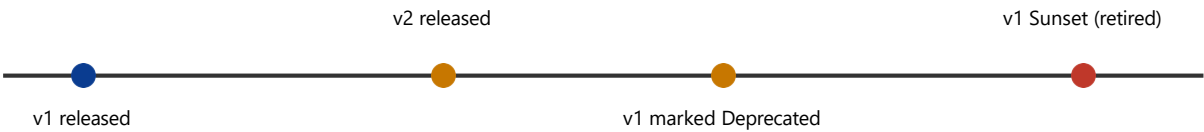
Build APIs that change safely without breaking consumers.

10.1 Breaking vs. Additive Changes

Additive (safe, no version bump)	Breaking (requires new major version)
New optional field in response	Removing/renaming a field
New optional query parameter	Changing a field's type
New endpoint	Changing default/required-ness of a field
New enum value <i>if consumers are told to expect unknowns</i>	Removing an endpoint or status code behavior

10.2 Deprecation Headers & Timeline

```
Deprecation: true
Sunset: Wed, 31 Dec 2026 23:59:59 GMT
Link: <https://api.example.com/docs/migration/v2>; rel="deprecation"
```



10.3 Retiring a Version Gracefully

- Announce the sunset date at least one full deprecation window in advance (per handbook policy, e.g. 6–12 months).
- Monitor traffic on the deprecated version; contact remaining consumers directly before cutoff.
- After sunset, return `410 Gone` rather than 404, so consumers know it existed and was retired on purpose.

10.4 Versioned Routing in FastAPI

```

v1_router = APIRouter(prefix="/v1")
v2_router = APIRouter(prefix="/v2")

@v1_router.get("/orders/{order_id}")
async def get_order_v1(order_id: str, response: Response):
    response.headers["Deprecation"] = "true"
    response.headers["Sunset"] = "Wed, 31 Dec 2026 23:59:59 GMT"
    return legacy_shape(await service.get_order(order_id))

@v2_router.get("/orders/{order_id}")
async def get_order_v2(order_id: str):
    return await service.get_order(order_id)

app.include_router(v1_router)
app.include_router(v2_router)

```

10.5 Cache-Control for Sensitive Responses

```

@app.get("/account/{id}/balance")
async def get_balance(id: str, response: Response):
    response.headers["Cache-Control"] = "no-store, private"
    return await service.get_balance(id)

```

Sensitive or per-user data must never be cached by shared proxies — use `no-store, private`; public reference data can use `public, max-age=...`

API Quality, Testing & Operational Maturity

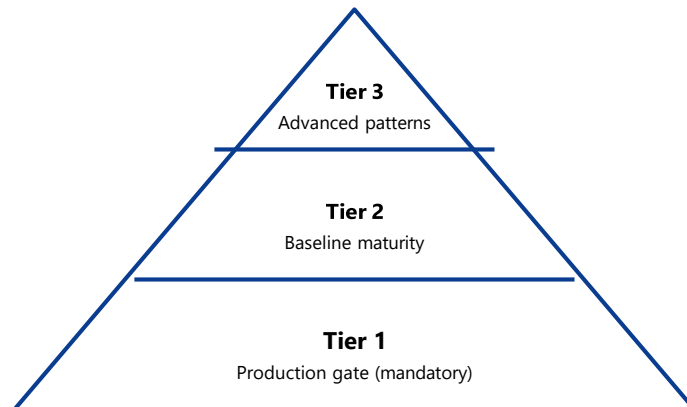
Prove the API is production-ready and keep it that way.

Module 11 — The Tiered Definition of Done

Learn the tiered self-assessment framework used to validate production readiness. Log aggregation via ELK Stack / Splunk; dashboards and alerting via Grafana.

11.1 What a Tiered Definition of Done Is

Instead of one binary "done," maturity is assessed across three progressive tiers — each a hard prerequisite for the next.



11.2 Tier 1 — Production Gate Requirements

Mandatory before any production traffic: authn/authz on every route, standard error envelope, request-ID tracing, health/readiness endpoints, structured logs with no sensitive data, and an up-to-date OpenAPI contract.

11.3 Tier 2 — Baseline Maturity Requirements

Metrics endpoint (request count/errors/latency), defined SLIs/SLOs, negative-path test coverage, versioning headers in place, pagination on all list endpoints.

11.4 Tier 3 — Advanced Pattern Requirements

Idempotency keys, async job pattern, rate-limit signaling, ETag-based optimistic concurrency, distributed tracing across services, error-budget-driven alerting (CMM Level 4–5, see Appendix A).

11.5 How to Conduct a Self-Assessment

Walk every route against the checklist below; record each item, don't skip ambiguous ones — mark them Partial and note why.

Module 12 — API Testing: Compliance & Negative Testing

Write tests that serve as compliance evidence for each quality tier.

12.1 FastAPI TestClient for In-Process Testing

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_get_order_success():
    resp = client.get("/v2/orders/ord_1", headers={"Authorization": "Bearer valid-token"})
    assert resp.status_code == 200
    assert resp.json()["id"] == "ord_1"
```

12.2 Organizing Tests by Quality Requirement

```
tests/
├── tier1_production_gate/
│   ├── test_auth_enforced.py
│   ├── test_error_envelope.py
│   └── test_health_endpoints.py
├── tier2_baseline/
│   ├── test_pagination.py
│   └── test_metrics_endpoint.py
└── tier3_advanced/
    ├── test_idempotency.py
    └── test_etag_concurrency.py
```

12.3 Compliance Test Patterns

```
def test_every_error_response_matches_envelope():
    resp = client.get("/v2/orders/does-not-exist", headers=AUTH)
    body = resp.json()
    assert "error" in body
    assert {"code", "message", "request_id"} <= body["error"].keys()

def test_response_matches_openapi_schema():
    resp = client.get("/v2/orders/ord_1", headers=AUTH)
    validate(instance=resp.json(), schema=load_schema("Order"))
```

12.4 Negative Test Categories

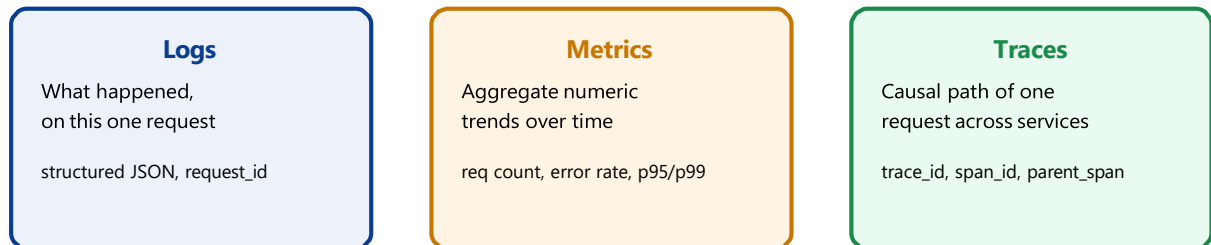
Category	Example
Missing/invalid auth	No header → 401; expired token → 401; wrong scope → 403

Malformed input	Wrong type, missing required field → 422 with details
Boundary values	quantity=0, limit=101, empty string id
Not found / gone	Unknown id → 404; retired resource → 410
Conflict	Duplicate create → 409; stale ETag on PUT → 412
Rate limiting	Burst requests → 429 with Retry-After

Module 13 — Observability: Logging, Metrics & Tracing

Make every API call visible, measurable, and traceable.

13.1 The Three Pillars of Observability



13.2 Structured JSON Logging with Required Fields

```
import logging, json, time
from starlette.middleware.base import BaseHTTPMiddleware

class JsonFormatter(logging.Formatter):
    def format(self, record):
        return json.dumps({
            "timestamp": self.formatTime(record, "%Y-%m-%dT%H:%M:%S%z"),
            "level": record.levelname,
            "message": record.getMessage(),
            "request_id": getattr(record, "request_id", None),
            "route": getattr(record, "route", None),
            "status_code": getattr(record, "status_code", None),
            "duration_ms": getattr(record, "duration_ms", None),
            "service": "orders-api",
        })

class AccessLogMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        start = time.perf_counter()
        response = await call_next(request)
        duration_ms = (time.perf_counter() - start) * 1000
        logger.info("request_completed", extra={
            "request_id": request.state.request_id,
            "route": request.url.path,
            "status_code": response.status_code,
            "duration_ms": round(duration_ms, 2),
        })
        return response
```

Required fields on every log line: `timestamp` , `level` , `request_id` , `route` , `status_code` , `duration_ms` , `service` .

13.3 Log Safety & Sensitive Data Protection

```
SENSITIVE_KEYS = {"password", "token", "authorization", "ssn", "card_number"}

def redact(payload: dict) -> dict:
    return {k: ("**REDACTED**" if k.lower() in SENSITIVE_KEYS else v) for k, v in payload.items()}
```

13.4 Request, Error & Duration Metrics Endpoint

```
from prometheus_client import Counter, Histogram, generate_latest
from fastapi import Response

REQUEST_COUNT = Counter("http_requests_total", "Total requests", ["method", "route", "status"])
REQUEST_LATENCY = Histogram("http_request_duration_seconds", "Latency", ["route"])
ERROR_COUNT = Counter("http_errors_total", "5xx/4xx errors", ["route", "status"])

class MetricsMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        with REQUEST_LATENCY.labels(route=request.url.path).time():
            response = await call_next(request)
            REQUEST_COUNT.labels(request.method, request.url.path, response.status_code).inc()
            if response.status_code >= 400:
                ERROR_COUNT.labels(request.url.path, response.status_code).inc()
            return response

@app.get("/metrics")
async def metrics():
    return Response(generate_latest(), media_type="text/plain")
```

13.5 Distributed Tracing with Standard Trace Headers



traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01

Single trace_id propagated across all 3 spans →
stitched into one waterfall view in the tracing backend (Jaeger/Tempo)

```

from opentelemetry import trace
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor

FastAPIInstrumentor.instrument_app(app)    # auto-creates spans, propagates W3C traceparent header

tracer = trace.get_tracer("orders-api")

@app.get("/orders/{order_id}")
async def get_order(order_id: str):
    with tracer.start_as_current_span("fetch_order_from_db"):
        return await repo.get(order_id)

```

13.6 Log Aggregation & Metrics Platform Integration

Platform	Role	Integration pattern
ELK Stack (Elasticsearch/Logstash/Kibana)	Centralized log search & dashboards	JSON logs to stdout → Filebeat/Fluentd ships to Logstash → indexed in Elasticsearch
Splunk	Enterprise log analytics & alerting	Universal Forwarder or HEC (HTTP Event Collector) ingest of JSON logs
Grafana	Dashboards & alerting on metrics/logs/traces	Queries Prometheus (metrics), Loki (logs), Tempo/Jaeger (traces)

Module 14 — Advanced Patterns & Long-Term Maturity

Explore the patterns that take an API beyond baseline compliance.

14.1 Idempotency Keys on Sensitive Operations

```
@app.post("/payments")
async def create_payment(payload: PaymentCreate, idempotency_key: str = Header(...)):
    existing = await idempotency_store.get(idempotency_key)
    if existing:
        return existing.response_body # replay original result, no double-charge
    result = await payments_service.charge(payload)
    await idempotency_store.save(idempotency_key, result)
    return result
```

14.2 Asynchronous Job Pattern

```
@app.post("/reports", status_code=202)
async def start_report(payload: ReportRequest):
    job_id = await jobs.enqueue(payload)
    return {"job_id": job_id, "status_url": f"/reports/jobs/{job_id}"}

@app.get("/reports/jobs/{job_id}")
async def get_job_status(job_id: str):
    job = await jobs.get(job_id)
    return {"status": job.status, "result_url": job.result_url if job.status == "done" else None}
```

14.3 Rate Limit Signaling

```
@app.middleware("http")
async def rate_limit_headers(request, call_next):
    response = await call_next(request)
    response.headers["X-RateLimit-Limit"] = "100"
    response.headers["X-RateLimit-Remaining"] = str(remaining_for(request))
    response.headers["X-RateLimit-Reset"] = str(reset_epoch())
    return response
```

On 429, always include `Retry-After` so clients back off correctly instead of retrying immediately.

14.4 Cache-Control in Depth

Directive	Meaning
<code>public, max-age=3600</code>	Shared/CDN cacheable for 1 hour
<code>private, max-age=60</code>	Only the end-user's client may cache
<code>no-store</code>	Never persist the response anywhere

stale-while-revalidate=30

Serve stale up to 30s while refreshing in background

14.5 Optimistic Concurrency with Entity Tags (ETags)

```
@app.get("/orders/{id}")
async def get_order(id: str, response: Response):
    order = await repo.get(id)
    response.headers["ETag"] = order.version_hash
    return order

@app.put("/orders/{id}")
async def update_order(id: str, payload: OrderUpdate, if_match: str = Header(None, alias="If-Match")):
    current = await repo.get(id)
    if if_match != current.version_hash:
        raise HTTPException(status_code=412, detail="PRECONDITION_FAILED")
    return await repo.update(id, payload)
```

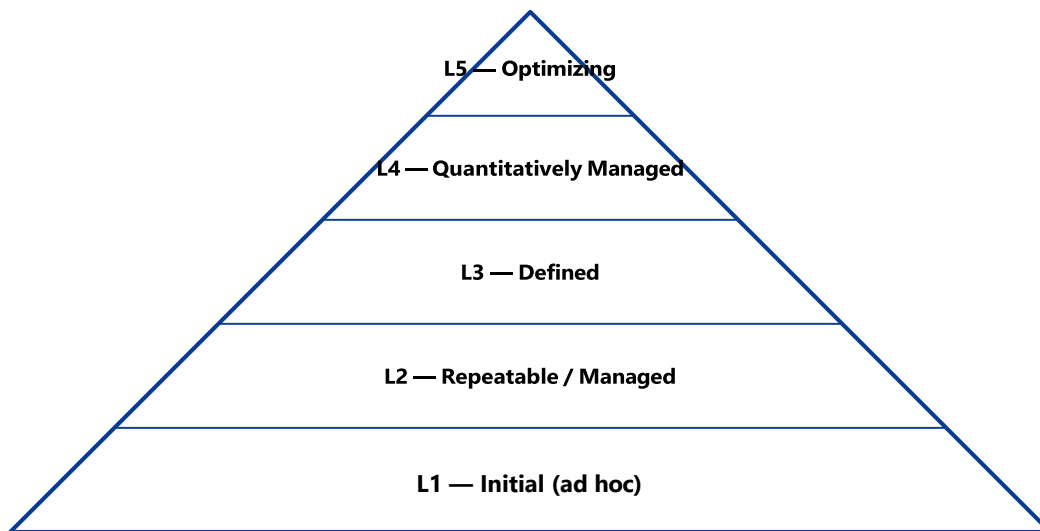
ETags prevent silent lost updates when two clients edit the same resource concurrently — the second writer's stale ETag is rejected with 412.

Appendix A —

Observability & SLO API Testing: CMM Levels 1–5

Hands-on exercises mapping the Capability Maturity Model to API observability & SLO practice.

A.1 CMM Maturity Pyramid Applied to Observability



CMM Level	Observability characteristic	SLO / testing practice
L1 — Initial	No structured logs, no metrics; debugging via <code>print()</code> or manual server access	No SLOs defined; readiness checked manually before releases
L2 — Repeatable/Managed	Basic access logs exist; a <code>/health</code> endpoint is present	Informal targets ("should be fast"); manual smoke tests after deploy
L3 — Defined	Standardized structured JSON logs w/ request ID; <code>/metrics</code> endpoint; log/metric schema documented org-wide	SLIs formally defined (latency, error rate, availability); SLOs documented and reviewed; automated compliance tests in CI
L4 — Quantitatively Managed	Dashboards track SLIs continuously; distributed tracing across services; alerting on SLO burn rate	Error budgets calculated and tracked; automated SLO regression tests gate deploys; historical trend analysis
L5 — Optimizing	Predictive/anomaly-based alerting; auto-remediation; chaos/fault-injection testing; continuous tuning of instrumentation	SLO-driven progressive delivery (auto-rollback on burn-rate breach); continuous improvement loop feeding design decisions

A.2 Exercise 1 (Level 1 → 2): Add a Health Endpoint & Basic Access Log

Goal: move off "ad hoc" — give the API a health signal and a consistent access log line for every request.

```

@app.get("/healthz")
async def healthz():
    return {"status": "ok"}

# test: prove the signal exists and is fast
def test_health_endpoint_returns_ok_under_100ms():
    start = time.perf_counter()
    resp = client.get("/healthz")
    elapsed_ms = (time.perf_counter() - start) * 1000
    assert resp.status_code == 200
    assert elapsed_ms < 100

```

A.3 Exercise 2 (Level 2 → 3): Structured Logging + SLI Definition

Goal: emit structured JSON logs with request_id/route/status/duration, and formally define SLIs/SLOs for the service.

```

# SLI/SLO definition (checked into repo as slo.yaml)
service: orders-api
slis:
  - name: availability
    definition: "successful_requests / total_requests"
  - name: latency_p95
    definition: "95th percentile of request duration"
slos:
  - sli: availability
    target: 99.9% # over rolling 30 days
  - sli: latency_p95
    target: "< 300ms"

# test: every response includes a request id, and logs are structured JSON
def test_response_includes_request_id_header():
    resp = client.get("/v2/orders/ord_1", headers=AUTH)
    assert "X-Request-ID" in resp.headers

def test_log_line_is_valid_json_with_required_fields(caplog):
    client.get("/v2/orders/ord_1", headers=AUTH)
    record = json.loads(caplog.records[-1].message)
    assert {"request_id", "route", "status_code", "duration_ms"} <= record.keys()

```

A.4 Exercise 3 (Level 3): Metrics Endpoint & Automated SLO Compliance Test

Goal: expose Prometheus-format metrics and write an automated test that asserts the service currently meets its documented SLO under a synthetic load sample.

```
def test_p95_latency_meets_slo_under_sample_load():
    durations = []
    for _ in range(50):
        start = time.perf_counter()
        resp = client.get("/v2/orders/ord_1", headers=AUTH)
        durations.append((time.perf_counter() - start) * 1000)
        assert resp.status_code == 200
    durations.sort()
    p95 = durations[int(0.95 * len(durations))]
    assert p95 < 300, f"p95 latency {p95}ms breaches 300ms SLO"

def test_metrics_endpoint_exposes_required_series():
    resp = client.get("/metrics")
    body = resp.text
    assert "http_requests_total" in body
    assert "http_request_duration_seconds" in body
    assert "http_errors_total" in body
```

A.5 Exercise 4 (Level 4): Error Budget Calculation & Burn-Rate Test

Goal: compute the error budget from the availability SLO and write a test/alert rule that fails when burn rate is too fast to hit the monthly target.

```
def error_budget_remaining(slo_target: float, total: int, errors: int) -> float:
    allowed_error_rate = 1 - slo_target          # e.g. 0.001 for 99.9%
    allowed_errors = total * allowed_error_rate
    return max(0.0, (allowed_errors - errors) / allowed_errors) if allowed_errors else 0.0

def test_error_budget_not_exhausted_this_window():
    total, errors = query_metrics_window("orders-api", days=30)
    remaining = error_budget_remaining(slo_target=0.999, total=total, errors=errors)
    assert remaining > 0.0, "error budget exhausted - halt risky deploys until budget recovers"

def test_burn_rate_alert_fires_on_fast_budget_consumption():
    # fast-burn alert: consuming budget 14x faster than sustainable over a 1h window
    burn_rate = current_hour_error_rate() / (1 - 0.999)
    assert burn_rate < 14.4, f"burn rate {burn_rate}x - page on-call"
```

A.6 Exercise 5 (Level 5): SLO-Gated Progressive Delivery & Chaos Test

Goal: demonstrate the "optimizing" behavior — deployments auto-gate on live SLO burn rate, and resilience is continuously verified via fault injection.

```
def test_canary_promotion_blocked_when_burn_rate_high(monkeypatch):
    monkeypatch.setattr("deploy.metrics.current_burn_rate", lambda: 20.0)
    decision = deploy.evaluate_canary_promotion(service="orders-api")
    assert decision == "ROLLBACK"

def test_service_degrades_gracefully_when_downstream_payments_api_is_slow(monkeypatch):
    monkeypatch.setattr(payments_client, "charge", lambda *a: (_ for _ in ()).throw(TimeoutError))
    resp = client.post("/v2/orders", json=VALID_ORDER, headers=AUTH)
    # circuit breaker should return 503 fast, not hang the whole request
    assert resp.status_code == 503
    assert resp.json()["error"]["code"] == "DOWNSTREAM_UNAVAILABLE"
```

A.7 Self-Assessment Checklist — CMM Levels for Observability

Level	Checklist item	Pass / Fail / Partial / N/A
1	Health endpoint exists	
2	Structured logs with request_id on every route	
3	SLIs/SLOs documented; /metrics endpoint live; CI compliance tests	
4	Error budget tracked; burn-rate alerting configured	
5	SLO-gated deploys; chaos/fault-injection tests in the suite	

Appendix B — Quick Reference Tables

B.1 Error Code Registry

HTTP	error.code
400	VALIDATION_ERROR
401	UNAUTHENTICATED
403	FORBIDDEN
404	NOT_FOUND
409	CONFLICT
410	GONE
412	PRECONDITION_FAILED
422	VALIDATION_ERROR
429	RATE_LIMITED
500	INTERNAL_ERROR
503	DOWNSTREAM_UNAVAILABLE / NOT_READY

B.2 Required Headers

Header	Purpose
X-Request-ID	End-to-end request correlation
traceparent	W3C distributed trace propagation
Deprecation / Sunset	Versioning lifecycle signaling
ETag / If-Match	Optimistic concurrency
Idempotency-Key	Safe retries of non-idempotent POSTs
Cache-Control	Caching policy (public/private/no-store)
X-RateLimit-*	Rate limit signaling

B.3 Tiered Definition of Done — Summary

Tier	Focus
1 — Production Gate	Auth, errors, health, logging, contract
2 — Baseline Maturity	Metrics, SLIs/SLOs, negative tests, pagination, versioning
3 — Advanced Patterns	Idempotency, async jobs, ETags, tracing, error budgets

---- END ----